

IMPROVER²: ITERATIVELY SELF-IMPROVING SLMs FOR NEUROSymbOLIC PROOF OPTIMIZATION

Anonymous authors

Paper under double-blind review

ABSTRACT

[Does this violate the double-blind criterion? Can we refer to previous work if we don't acknowledge that it is ours?]

The body of formalized mathematics has been growing steadily. From beginners to experts, when users finish a proof, they invariably revise it to improve it with respect to various criteria, such as length, readability, maintainability, or style. Our goal is to automate this process. Given the severe shortage of formal training data, one might also want to rewrite existing proofs for training purposes, not only to provide more data, but also to optimize that data with respect to the kinds of proofs we want models to generate.

Previous work [cite] on rewriting proofs suffers from shortcomings. First, it relies on a commercial foundation model, so the training is slow, expensive, and opaque. Second, task-time performance was also slow and expensive, limiting its utility. Finally, the metrics used to evaluate the model are simplistic and not clearly aligned to user goals.

To overcome these shortfalls, we introduce ImProver²: a neurosymbolically augmented, bootstrapped RL pipeline that iteratively trains small language models (SLMs) to optimize research-grade Lean4 proofs against arbitrary user-defined metrics. Our system extracts formal signals (context, goals, dependencies) and leverages auto-informalized structure and related lemmas to guide proof search. We iterate this neurosymbolic search with a stable, high-throughput learning loop that mixes a weighted SFT variant with IRPO, outperforming standard SFT and DPO training loops. Integrated into an expert-iteration framework, ImProver² self-improves from a cold start without external data or expert models; when available, distillation further boosts SLMs, enabling them to surpass much larger models on optimization tasks. We evaluate ImProver² on real-world research-level proofs, and across both formal and informal optimization objectives – length minimization, modularity maximization, dependency minimization, and readability maximization — demonstrating robust and practical optimizations that assist in proof structure and overall library quality, additionally empirically showing that training simple neural theorem provers on such structure-optimized proofs yields significant downstream improvements, underscoring proof optimization as a key lever for scalable formal mathematics.

1 INTRODUCTION

Formal proof assistants – such as Lean, Rocq, and Isabelle – have transformed mathematical practice by making correctness explicit and mechanized. Community libraries (e.g., Lean’s Mathlib) now grow at a pace driven both by increasing human contributions and by recent advances in neural theorem proving and autoformalization. This rapid expansion is a double-edged sword: it accelerates formal coverage, but also stresses maintainability, pedagogy, and the long-term usability of libraries whose proofs are often heterogeneous in style and quality.

As these libraries scale, data quality becomes a central concern for two reasons. First, maintainers and downstream users benefit from proofs that are concise, modular, and readable, rather than opaque tactic scripts that obscure structure. Second, better library quality increases their utility as training data: modern theorem provers and autoformalizers increasingly train on these very corpora,

so the quality, structure, and readability of proofs directly shape downstream prover performance. We corroborate this second point empirically by showing that provers trained on structure-optimized corpora outperform those trained on unoptimized data.

Unfortunately, the growth rate of formal libraries already exceeds what human reviewers and maintainers can reliably curate. Moreover, as the capabilities of autoformalizers improve, the volume of machine-generated content included in these corpora will only continue to grow – and as we already can see from recent autoformalization and neural theorem proving results, the guarantees of proof correctness provided by these formal systems do not carry any similar guarantees as to the proof quality, modularity, or understandability.

This motivates automated proof optimization: given a verified proof, produce a proof that is better with respect to a user-defined objective (e.g., shorter length, greater modularity/declarativity, fewer dependencies, or higher readability), while preserving the underlying correctness of the proof as verified by the formal system. Crucially, the optimization objective is task- and user-dependent, and may be partly informal – therefore, any practical solution must support arbitrary metrics and scale to research-level theorems.

Prior work introduced this problem setting and demonstrated that large, state-of-the-art large-language models (LLMs) can improve simple metrics via agentic prompting and retrieval. However, these systems underperform on complex, research-grade theorems and richer objectives, and their inference cost makes them impractical at library scale. Moreover, performance gains tend to rely on larger base models and higher sampling budgets, which further limits their accessibility to the formal mathematics community.

We address these limitations with **ImProver²**, a neurosymbolically augmented, bootstrapped reinforcement learning pipeline that trains small language models (SLMs) to optimize Lean proofs under arbitrary, user-defined metrics. Our approach extracts rich formal signals (context, goals, dependencies) and integrates auto-informalized structure and related lemmas to focus search. Training proceeds via an expert-iteration loop that mixes a sample-efficient variant of supervised fine-tuning (weighted by measured improvement) with an RL objective (IRPO) tailored to metric deltas, enabling stable self-improvement from a cold start. When available, the pipeline can incorporate distillation, allowing SLMs to match or surpass much larger models on optimization tasks.

As such, our primary contributions are:

1. *Self-improving SLMs that surpass larger models.* We demonstrate a data-free, expert-iteration pipeline that bootstraps small language models (SLMs) to significantly multiply performance on highly specialized research libraries. Moreover, given an initial distillation step, these SLMs can outperform much larger models on research-level proof optimization tasks, at near-zero compute cost.
2. *Novel and practical optimization metrics.* We define a suite of formal metrics (length, modularity/declarativity via spawned-goal counts, dependency footprint) and a stable informal metric (LLM-as-judge readability) designed for reliable, pairwise evaluation under randomization and consensus.
3. *A highly stable iterative RL pipeline.* We present a robust expert-iteration loop with weighted SFT (wSFT) and an IRPO (DPO with explicit NLL regularization) stage, enabling data-free self-improvement with strong stability and sample efficiency.
4. *Neurosymbolic augmentation for research mathematics.* We introduce Lean-aware extraction, goal-state Chain-of-States (CoS) annotation, auto-informalization of statements and tactic steps, and premise retrieval over a library-wide index to guide structure-aware optimization.

2 RELATED WORK

TODO

3 PROOF OPTIMIZATION

Given a verified proof of a theorem, a proof optimization agent’s objective is to synthesize a semantically equivalent proof that is “better”, according to a user-specified objective, while remaining verifiably correct according to the Lean kernel. Objectives may be single- or multi-criteria and can include informal components (e.g., readability).

3.1 SETUP AND NOTATION

More formally, let $\mathcal{C}$ denote contexts (imports, local declarations, module metadata, etc.), $\mathcal{X}$ theorem statements, and $\mathcal{Y}$ Lean proofs. An instance is $(c, x, y) \in \mathcal{C} \times \mathcal{X} \times \mathcal{Y}$, where y is a proof of x in c , though we have no guarantees about the correctness and compilation of this theorem and proof yet.

As such, the Lean kernel induces

$$\ker(c, x, y) = \mathbb{I}\{\text{lean_check}(c, x, y) = \top\} \in \{0, 1\},$$

$$\mathcal{F}(c, x) = \{y \in \mathcal{Y} : \ker(c, x, y) = 1\}.$$

Two proofs $y, y' \in \mathcal{F}(c, x)$ are *semantically equivalent* if both prove x in c .

3.2 OPTIMIZATION METRICS

Optimization metrics quantify the quality of a proof y of x in context c , and serve as the basis for user-defined objectives and preferences in proof optimization. Namely, we define two possible classes of optimization metrics:

- A relative metric is a (possibly vector-valued) map:

$$\Delta : \mathcal{C} \times \mathcal{X} \times \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}^k,$$

- An absolute metric is a (possibly vector-valued) map:

$$\mu : \mathcal{C} \times \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}^k,$$

with coordinates capturing distinct desiderata. Note that absolute metrics induce a relative metric via:

$$\Delta_\mu(c, x; y, y_0) = \mu(c, x, y) - \mu(c, x, y_0).$$

In practice, we postcompose a scalarization mapping to our relative metrics to obtain a single score, representing the *improvement* of a proof y compared to a baseline proof y_0 of a theorem x in c . Namely, given a relative metric Δ and a scalarization mapping $\phi : \mathbb{R}^k \rightarrow \mathbb{R}$, we define:

$$\Delta_\phi(c, x; y, y_0) = \phi(\Delta(c, x; y, y_0)).$$

Given an absolute metric, we can similarly define the scalarization of its induced relative metric via $\Delta_{\phi, \mu}(c, x; y, y_0) = \phi(\Delta_\mu(c, x; y, y_0))$.

In practice, especially in the case where $\phi : \mathbb{R} \rightarrow \mathbb{R} = \text{id}$, we will oftentimes omit the scalarization ϕ for the sake of brevity and simply write $\Delta_\mu(c, x; y, y_0)$ to denote the scalarized relative metric.

With this, we seek a verified candidate with maximal improvement while excluding pathological edits. Let $\mathcal{A}_\Delta(c, x) \subseteq \mathcal{Y}$ be the *admissible* set that filters degeneracies (e.g., proof-term no-ops, vacuous have chains, blind inlining explosions) parameterized by the metric in question Δ . Then, our objective is

$$\max_{\substack{y \in \mathcal{A}_\Delta(c, x) \\ \ker(c, x, y) = 1 \\ \ker(c, x, y_0) = 1}} \Delta_\phi(c, x; y, y_0) \quad (1)$$

In practice we solve equation 1 via conditional generation followed by verification and selection.

3.2.1 METRICS OF INTEREST

In this work, we focus and evaluate on a collection of four canonical metrics, which are designed to be practical, interpretable, and automatable at scale.

- **Length:** We aim to minimize the length of proofs, measured by the number of tactics. In practice, proof golfing is a common activity in formal mathematics, and shorter proofs are prized as being often easier to read and maintain. As such, we can define the absolute length metric $\mu_{len} : \mathcal{C} \times \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ as the negated normalized tactic count:

$$\mu_{len}(c, x, y) = -\text{num_tactics}(y)$$

Additionally, note that admissibility can be defined to exclude proof-term conversions, which convert the length to 0 without changing the semantics of the proof, and oftentimes decreasing the readability, maintainability, and overall quality of the proof. In other words, define:

$$\mathcal{A}_{len}(c, x) = \{y \in \mathcal{Y} \mid \mu_{len}(c, x, y) > 0\}$$

- **Modularity:** We aim to maximize the modularity of proofs, which we intuitively define as the number of independent subproofs in our proof. Namely, we deconstruct the proof into a tree of tactics, with edges representing – at a high level – the logical dependencies and flow between tactics in the proof. In this tree, we mark a subset of edges as “spawned” if they correspond to goals that are introduced by the proof but are not direct subgoals of the current tactic, which naturally arise from tactics like `have`, `nested by/ calc`, etc – and correspond with the informal notion of “modular” independent subproofs.

To formally do this, we must first define the notion of a spawned goal.

Fix a theorem in context (c, x) and a verified proof $y \in \mathcal{F}(c, x)$. The Lean elaborator induces a finite sequence of *proof states*

$$(M_i, \mathcal{A}_i) \xrightarrow{\tau_i} (M_{i+1}, \mathcal{A}_{i+1}) \quad i = 1, \dots, T,$$

where $M_i = (\Delta_i, \sigma_i)$ is a metavariable context (partial assignment σ_i over declarations Δ_i) and $\mathcal{A}_i \subseteq \text{Unassigned}(M_i)$ is the active goal list.

Goals and children: A goal is an unassigned metavariable $?m$ with declaration $(\Gamma_{?m}, A_{?m})$ in Δ_i . Each step τ_i *focuses* a goal $\bar{g}_i \in \mathcal{A}_i$ and assigns it a term $\sigma_{i+1}(\bar{g}_i) = t_i$ with $\Gamma_{\bar{g}_i} \vdash t_i : A_{\bar{g}_i}$. The *direct children* (subgoals) of τ_i are

$$\text{Children}(\tau_i) = \{?m \mid ?m \in \text{Unassigned}(M_{i+1}) \text{ and } ?m \text{ occurs free in } t_i\}.$$

Step graph (proof tree over steps): Let $V = \{1, \dots, T\}$ index steps. We draw a (normal) edge $i \rightarrow j$ iff $\bar{g}_j \in \text{Children}(\tau_i)$; roots are those i whose $\bar{g}_i$ has no incoming normal edge. This yields a rooted forest (V, E) over *steps* rather than metavariables.

Spawned goals: Not all obligations that surface at time i are produced as children of $\bar{g}_i$. Let the *currently surfaced* set be

$$\text{Current}_i = \{\bar{g}_i\} \cup \text{Children}(\tau_i).$$

Let $S_1 = \emptyset$ and $S_{i+1} = S_i \cup \text{Children}(\tau_i)$ be the set of children seen so far. We define the *spawned goals at step i* by

$$\text{Spawned}(\tau_i) = (\text{Current}_i \setminus \text{Children}(\tau_i)) \setminus S_i.$$

Thus $\text{Spawned}(\tau_i)$ are the goals that *first appear* at step i but are *not* direct children of $\bar{g}_i$ (e.g., obligations surfaced by `have`, `nested by/ calc`, `automation`, or `focus/defocus`). We record these as distinguished edges $i \rightsquigarrow j$ whenever $\bar{g}_j \in \text{Spawned}(\tau_i)$, yielding a labeled forest (V, E, E_{sp}) with outgoing edges partitioned into *normal* and *spawned*.

Modularity metric. Note that given a proof y of a theorem x in context c , we can extract the steps $(\tau_i)_{i=1}^T$, and thereby define the absolute modularity metric $\mu_{mod} : \mathcal{C} \times \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ as:

$$\mu_{mod}(c, x, y) = \sum_{i=1}^T |\text{Spawned}(\tau_i)|$$

Intuitively, this scores the proof by the total number of "declarative" or "modular" components it contains, as measured by the number of spawned goals. This encourages proofs that break down complex arguments into smaller, reusable subproofs, which are often easier to read and maintain.

Finally, we can define admissibility to exclude proofs that adversarially spawn many trivial goals without making progress. **TODO FINISH DESCRIBING ADMISSIBILITY CRITERIA - DISALLOW TRIVIAL SPAWN CHAINS?**

- **Dependencies:** We aim to minimize the dependency footprint of proofs, measured by the number of unique external lemmas explicitly used in the proof. This encourages self-contained proofs that do not rely on a large number of auxiliary results, which can improve readability and maintainability.

We explicitly define the absolute dependency metric $\mu_{dep} : \mathcal{C} \times \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ as:

$$\mu_{dep}(c, x, y) = -|\{(c', x', y') \in \text{Deps}(c, x, y) : \text{type}(x') = \text{theorem}\}|$$

The `Deps` function is defined to be the set of all external dependencies, including but not limited to lemmas. It is described in more detail in §4.2.1. Additionally, like the length metric, we multiply by the logarithm of the kernel check to ensure that incorrect proofs receive the worst possible score, $-\infty$, and verified proofs receive a score equal to the negated number of unique dependencies.

We can define admissibility to exclude proofs that trivially minimize dependencies by inlining all referenced lemmas, resulting in a proof that is correct but not maintainable or readable. Specifically, we can define:

$$\begin{aligned} \mathcal{A}_{dep}(c, x) &= \{y \in \mathcal{Y} \mid y \text{ does not inline dependencies}\} \\ &= \{y \in \mathcal{Y} \mid \forall (c', x', y') \in \text{Deps}(c, x, y), \text{type}(x') = \text{theorem} \implies \neg y.\text{contains}(y')\} \end{aligned}$$

- **Readability:** As a proof of concept for informal metrics, we aim to maximize the readability of proofs, measured by a consensus of LLM judgments over a fixed rubric. Readability is a subjective and multifaceted concept, but generally refers to how easily a human reader can understand and follow the logical flow of a proof.

Readability is scored based on a rubric derived from existing best practices in proof-writing [cite mathlib style guide]. We identify 7 main criteria for readability:

- Clarity: a clear, well-organized mathematical argument is developed by the proof. Similar ideas and supporting lemmas are grouped together; intermediate steps are organized into `have` statements.
- Use of outside theorems: the proof does not attempt to re-prove already proven results, utilizing previous theorems instead.
- Comments: the proof's argument is documented in natural language.
- Automation: the proof utilizes high-level automation tactics judiciously, and avoids overusing low-level instructions when they are not needed.
- Layout: the proof uses appropriate line breaks, whitespace, and indentation to split up steps or tactics; each line doesn't exceed around 100 characters.
- Variable conventions: the proof adheres to Mathlib-style variable naming conventions (α, β, γ for types, h, h^1 , and so on for hypotheses, etc.)
- Brevity: the proof doesn't use unnecessary tactics or terms.

This metric utilizes an LLM-as-a-judge framework [cite] to evaluate pairs of proofs (y_1, y_2) based on the above criteria. For each criterion c , k separate instances of an un-tuned language model are prompted with a detailed description of said criterion along with examples of proofs that do and don't conform to it; the model is then given both proofs in a randomized order and requested to determine whether y_1 is substantially better, approximately the same, or strictly worse in this category relative to y_2 (denoted $M_{c,i}(y_1, y_2) = 1$, $M_{c,i}(y_1, y_2) = 0$, and $M_{c,i}(y_1, y_2) = -1$ respectively). The relative readability metric is then defined as:

$$\Delta_{\mu \text{ readability}}(c, x; y_1, y_2) = \sum_{c=1}^7 w_c \sum_{i=1}^k \frac{M_{c,i}(y_1, y_2)}{k}$$

where $w \in \mathbb{R}^c$ are weights corresponding to each category.

To prevent reward-hacking, we additionally apply three heuristics to proofs scored in this way. First, before being scored for any criterion except for “Comments”, the comments of a proof are removed to disallow consideration of any non-symbolic and thus irrelevant material. Second, proofs that are identical apart from added and unnecessary trailing tactics are automatically scored as “the same” regardless of model output to prevent scoring based on the semblance of greater sophistication that has no bearing on the actual functionality of the proof. Finally, proof pairs where one member is a proof term t and the other’s proof consists solely of `by exact t` are scored as “the same” to prevent incentivization of trivial improvements.

TODO FINISH READABILITY VARIANCE ABLATIONS

3.3 GENERATOR

Score and objective. Given a baseline proof $y_0 \in \mathcal{F}(c, x)$, define the (*per-candidate*) *improvement score*

$$S_{\Delta}(c, x; y \| y_0) = \Delta(c, x; y, y_0) \cdot \ker(c, x, y) \cdot \ker(c, x, y_0) \cdot \mathbb{I}\{y \in \mathcal{A}_{\Delta}(c, x)\}$$

By construction of our metrics (§3.2), $S_{\Delta}(c, x; y \| y_0)$ is well-defined for all $y \in \mathcal{Y}$ and collapses non-compiling candidates to a score of 0 so that non-verifying y never win selection (or are indistinguishable from the $y = y_0$ case).

Thus, the optimization problem in equation 1 becomes

$$\max_{y \in \mathcal{Y}} S_{\Delta}(c, x; y \| y_0),$$

as we know that $y_0 \in \mathcal{A}_{\Delta}(c, x)$. In practice, we approximate this optimization problem by sampling and selecting.

Conditional generator. Let G_{θ} be a conditional distribution over proofs parameterized by θ and conditioned on an augmented description of the instance (c, x, y_0) via the mapping Ψ (§4.2), as well as a (serialized description of a) metric Δ :

$$y \sim G_{\theta} \left(\cdot \mid \underbrace{\Psi(c, x, y_0)}_{\text{neurosymbolic augmentation}}, \Delta \right)$$

Here, Ψ extracts out lean a program that exposes Lean-aware signals (context slice, goal-state traces, auto-informalized statement/steps, retrieved premises). Intuitively, Ψ factors the raw inputs (c, x, y_0) through canonical symbolic channels before they are serialized for the model. Namely, Ψ outputs additional channels of extracted data (serialized into strings) for context and formal dependencies, Chain-of-States augmentation, autoinformalized representations of y_0 , and potentially useful lemmas for x retrieved from c . Conditioning on Ψ, Δ therefore factors the generator through these neurosymbolically-augmented channels and a metric description rather than the raw triplet itself.

Best-of- n selection. Draw n i.i.d. samples $Y_1, \dots, Y_n \sim G_{\theta}(\cdot \mid \Psi, \Delta)$ and select

$$\hat{y}_n = \arg \max_{\substack{i \in [n] \\ Y_i \in \mathcal{A}_{\Delta}(c, x)}} S_{\Delta}(c, x; Y_i \| y_0).$$

This is an order-statistics estimator of the argmax: $\mathbb{E}[\max_i S_{\Delta}(c, x; Y_i \| y_0)]$ is nondecreasing in n , so increasing n monotonically improves the expected selected score (subject to the model’s coverage). In practice, ties are broken deterministically (lexicographic by hash).

4 IMPROVER²

ImProver² is a Lean-aware neurosymbolic system and training pipeline for proof optimization tasks that (i) augments the generator’s conditional with programmatically extracted structure and retrieved premises, and (ii) bootstraps small language models (SLMs) via an iterative RL loop.

4.1 OVERVIEW

Given a set of problems $\mathcal{P}$, a base language model G_0 , and an optimization metric Δ , we aim to iteratively increase the mean improvement score S_Δ achieved by G_t across iterations t .

To do this, we first partition the fixed problems set $\mathcal{P}$ into train/test as $\mathcal{P}_{\text{train}} \cup \mathcal{P}_{\text{test}} = \mathcal{P}$. Formally, a problem T is defined as a triplet $(c_T, x_T, y_{T,0})$. At iteration t we:

1. Generate n candidates per train problem $T \in \mathcal{P}_{\text{train}}$ with the current generator $G_t(\cdot \mid \Psi, \Delta)$, forming the (no-replay) dataset:

$$\mathcal{S}_{\text{nr}}^{(t)} = \{(c_T, x_T, y_{T,0}, \{(y_{T,i}, \hat{s}_{T,i})\}_{i=1}^n)\}_T$$

where $\hat{s}_{T,i} = S_\Delta(c_T, x_T; y_{T,i} \parallel y_{T,0})$ (as defined in §3.3).

2. Then, we collect all previous datasets $\mathcal{S}_{\text{re}}^{(j)}$ for $j < t$, and combine them with $\mathcal{S}_{\text{nr}}^{(t)}$ to postprocess the raw dataset with a replay buffer by replacing a target fraction of entries in $\mathcal{S}_{\text{nr}}^{(t)}$ with items sampled from older $\{\mathcal{S}_{\text{nr}}^{(j)}\}_{j < t}$ under the policy in §4.3.1. This yields $\mathcal{S}_{\text{re}}^{(t)}$, which we process to build training datasets for wSFT ($\mathcal{S}_{\text{wSFT}}^{(t)}$) and IRPO ($\mathcal{S}_{\text{IRPO}}^{(t)}$).
3. With G_t as the base model, train on $\mathcal{S}_{\text{wSFT}}^{(t)}$ with weighted SFT to get $G_t^{(\text{wSFT})}$ (§4.3.2).
4. With $G_t^{(\text{wSFT})}$ as the base/ref model, train on $\mathcal{S}_{\text{IRPO}}^{(t)}$ with IRPO to get $G_t^{(\text{IRPO})}$ (§4.3.3). Define $G_{t+1} = G_t^{(\text{IRPO})}$.
5. Evaluate and record the mean improvement score of G_{t+1} (and optionally $G_t^{(\text{wSFT})}$ as well) on $\mathcal{P}_{\text{test}}$ via best-of- n (§3.3).

And repeat for iteration $t + 1, t + 2$, and so on.

4.2 NEUROSymbOLIC AUGMENTATION

As mentioned in 3.3, we factor our generator – which in this case takes the form of a language model – through a semi-deterministic map that exposes Lean-aware structure and dependencies at both a formal and informal level. This map Ψ extracts and serializes three additional channels of information from the raw input triplet $I = (c, x, y_0)$:

$$\Psi : I \mapsto \left(I, \underbrace{\Psi_{\text{ctx}}(I)}_{\text{context slice}}, \underbrace{\Psi_{\text{cos}}(I)}_{\text{goal-state traces}}, \underbrace{\Psi_{\text{inf}}(I)}_{\text{auto-informalization}} \right).$$

The generator (LLM/SLM) sees only the serialization of (Ψ, Δ) , so its conditional distribution effectively factors through these channels. Now, we describe each channel in turn.

4.2.1 CONTEXT

When working with proofs in high-dependency environments, it is likely that a given proof y_0 relies on many lemmas, definitions, and other formal objects defined in the context c . We aim to extract and serialize a minimal set of these objects to better inform our generation process.

Towards this, we consider the Lean4 concrete syntax tree (CST) and abstract syntax tree (AST). The CST preserves surface tokens and spans; the AST resolves names, binds variables, attaches types/kinds, and canonicalizes structure (e.g., constant/definition/theorem nodes). Context extraction uses both:

- CST view (textual reachability): harvest all surface identifiers and their source spans that occur in x and in the proof text of y_0 .
- AST view (semantic reachability): resolve those identifiers to fully qualified symbols under the language’s environment (imports, namespaces, instances, modules); record declaration categories (e.g., definition, theorem) and provenance (module/package).

and serialize each triple as a short snippet where goal lists are pretty-printed into comments adjacent to τ_i . This turns hidden kernel states into explicit cues the model can condition on.

4.2.3 AUTO-INFORMALIZATION

Ψ_{inf} exposes a compact, human-readable layer over the formal source content by paraphrasing and elaborating on formal artifacts into short natural-language (NL) sketches that models can condition on without losing semantic anchors. The goal is to provide a soft, fuzzy layer of abstraction that captures the “meaning” of formal items while being robust to syntactic variation and surface-level noise. We emphasize that this informalization is a secondary channel for the generator and serves simply as an additional representation of the target theorem; outputs are still prompted to be generated formally, and correctness is still judged formally.

Let $\mathcal{I}$ be an informalizer that maps a formal Lean theorem x – or a formal tactic step in a proof y_0 of x – into natural language (NL).

Then, given a statement x and a baseline proof y_0 with step indices $i \in \{1, \dots, T\}$ (aligned to CoS), we produce

$$\Psi_{\text{inf}}(x, y_0) = \left(\underbrace{\text{NL_stmt}(x)}_{\mathcal{I}(x)}, \underbrace{\langle \text{NL_step}_i \rangle_{i=1}^T}_{\text{NL_step}_i = \mathcal{I}(\tau_i: \text{goalsBefore}_i \rightarrow \text{goalsAfter}_i)} \right),$$

where each NL_step_i describes the tactic’s semantic effect (e.g., “case on whether $a < b$, $b < a$, or $a = b$ ”, “apply the commutativity of addition”, “consider an arbitrary n ”).

In practice, we instantiate $\mathcal{I}$ with the same base LLM/SLM as the generator, prompted once to emit both NL_stmt and per-tactic NL_step ; outputs are deterministically serialized and injected into prompts. We do not externally supervise this process, rather allowing the internal CoT chains of the underlying reasoning LLM to ensure the alignment. As such, there are no strong guarantees as to the “accuracy” of the informal outputs, rather, only the empirical results of their downstream utility in proof optimization as shown in §4.2.

4.3 TRAINING

4.3.1 DATASETS AND REPLAY BUFFER

On a fixed iteration t , after running inference with G_t (conditioned over Φ, Δ , as discussed in §4.2), we obtain the (no-replay) dataset

$$\mathcal{S}_{\text{nr}}^{(t)} = \left\{ (c_T, x_T, y_{T,0}, \mathcal{Y}_T^{(t)}) \mid T \in \mathcal{P}_{\text{train}} \right\}, \quad \mathcal{Y}_T^{(t)} = \{(y_{T,i}, \hat{s}_{T,i})\}_{i=1}^n.$$

where $\hat{s}_{T,i} = S_{\Delta}(c_T, x_T; y_{T,i} \parallel y_{T,0})$ (see §3.3).

We then construct the solutions (replay) dataset $\mathcal{S}_{\text{re}}^{(t)}$ by post-processing $\mathcal{S}_{\text{nr}}^{(t)}$ together with the previous iteration’s already post-processed dataset $\mathcal{S}_{\text{re}}^{(t-1)}$. The procedure is parameterized by a target replay proportion $\rho \in [0, 1]$, an improvement-rate cap $\pi_{\text{max}} \in (0, 1]$, and a `mode` $\in \{\text{mark}, \text{join}, \text{replace}\}$.

Additionally, for problem T , define its improvement rate at iteration j by

$$\pi_T^{(j)} = \frac{1}{n} \sum_{i=1}^n \mathbf{1} \left[\text{ker}(c_T, x_T, y_{T,i}^{(j)}) = 1 \wedge \hat{s}_{T,i}^{(j)} > 0 \right].$$

A problem is replay-eligible at iteration t iff it had at least one improved, compiling solution in some previous iteration $j < t$. We maintain a reservoir $\mathcal{E}$ of replay-eligible problems with preference for “easy” items (largest $\pi_T^{(j)}$ across $j < t$).

We proceed now to construct the post-replay buffer dataset in two main steps:

1. *Mark*: We first mark problems in $\mathcal{S}_{\text{nr}}^{(t)}$ as `REPLAY` or `FRONTIER` so that the fraction of replay equals ρ :

- (a) Start with the subset of problems that are replay-eligible; if their fraction exceeds ρ , downsample them by removing highest- π_T items (i.e. the “easiest”) until the replay fraction is ρ (cap at $\pi_{\max}$).
- (b) If their fraction is below ρ , replace uniformly chosen frontier items by items sampled from the reservoir $\mathcal{E}$ (taken from $\mathcal{S}_{\text{re}}^{(t-1)}$) until the replay fraction reaches ρ as best as possible. This keeps dataset size fixed while achieving the target mix.

After this step, every item in the dataset is tagged as REPLAY or FRONTIER and the target mix is satisfied.

2. Merge

For each item marked REPLAY with key $(c_T, x_T, y_{T,0})$, find its counterpart in $\mathcal{S}_{\text{re}}^{(t-1)}$, say with candidate (multi)set $\tilde{\mathcal{Y}}_T^{(t-1)}$. Then, we case on mode as follows:

- **join**: set the current candidates to the union $\tilde{\mathcal{Y}}_T^{(t)} = \mathcal{Y}_T^{(t)} \cup \tilde{\mathcal{Y}}_T^{(t-1)}$ (deduplicated by normalized proof text). This increases candidate diversity for both wSFT and IRPO.
- **replace**: set $\tilde{\mathcal{Y}}_T^{(t)} = \tilde{\mathcal{Y}}_T^{(t-1)}$, i.e., overwrite the current candidates by the previous iteration’s.

If mode is **mark**, skip this step (no candidate-level splice). In other words, set $\tilde{\mathcal{Y}}_T^{(t)} = \mathcal{Y}_T^{(t)}$

With this, we obtain the replay dataset:

$$\mathcal{S}_{\text{re}}^{(t)} = \left\{ (c_T, x_T, y_{T,0}, \tilde{\mathcal{Y}}_T^{(t)}) \mid T \in \mathcal{P}_{\text{train}} \right\}$$

4.3.2 WEIGHTED SUPERVISED FINE-TUNING (wSFT)

For each problem T (i.e. a tuple $(c_T, x_T, y_{T,0}, \tilde{\mathcal{Y}}_T^{(t)}) \in \mathcal{S}_{\text{re}}^{(t)}$), define the verified improvement set

$$\mathcal{K}_T^{(t)} = \left\{ i \in [\tilde{\mathcal{Y}}_T^{(t)}] : \ker(c_T, x_T, y_{T,i}) = 1 \wedge \hat{s}_{T,i} > 0 \right\} / \sim,$$

where $\sim$ deduplicates equal proof texts. If $\mathcal{K}_T^{(t)} = \emptyset$ the problem is excluded from wSFT (it will still enter IRPO via loser pairs).

Let $\{s_{T,i}\}_{i \in \mathcal{K}_T^{(t)}}$ denote the improvement scores on $\mathcal{K}_T^{(t)}$. With temperature $\tau_{\text{soft}} > 0$ and a trimming threshold $\epsilon_{\min} \geq 0$, define unnormalized weights

$$\tilde{w}_{T,i} = \exp(s_{T,i}/\tau_{\text{soft}}) \cdot \mathbf{1}[\exp(s_{T,i}/\tau_{\text{soft}}) \geq \epsilon_{\min}],$$

and normalize per problem:

$$w_{T,i} = \frac{\tilde{w}_{T,i}}{\sum_{j \in \mathcal{K}_T^{(t)}} \tilde{w}_{T,j}}, \quad \sum_{i \in \mathcal{K}_T^{(t)}} w_{T,i} = 1.$$

With this, we get a final training dataset for wSFT as:

$$\mathcal{S}_{\text{wSFT}}^{(t)} = \{(c_T, x_T, y_{T,0}, y_{T,i}, w_{T,i}) \mid i \in \mathcal{K}_T^{(t)}, T \in \mathcal{P}_{\text{train}}\}$$

In practice, we serialize $(c_T, x_T, y_{T,0})$ into a single string as the input prompt, and serialize $y_{T,i}$ as the output response.

Additionally, for a theorem T , let $\text{Var}_T = \text{Var}\{\hat{s}_{T,i} : i \in [n], \ker = 1\}$. We postprocess $\mathcal{S}_{\text{wSFT}}^{(t)}$ by keeping only the bottom q -quantile of theorems T given by Var_T , defaulting to $q = 0.8$. We do this so that the subsequent IRPO stage can receive the top $(1 - q)$ quantile of theorems as fresh data. This hyperparameter q is ablated in §5.

With this dataset $\mathcal{S}_{\text{wSFT}}^{(t)}$, we calculate the wSFT loss as:

$$\mathcal{L}_{\text{wSFT}}(\theta) = \mathbb{E}_T \left[\sum_{i \in \mathcal{K}_T^{(t)}} w_{T,i} \left(-\log G_\theta(y_{T,i} \mid \Psi(c_T, x_T, y_{T,0}), \Delta) \right) \right].$$

Here ℓ_{rat} is a standard next-token loss on the rationale tokens (masked if absent). The per-problem sum of weights equals 1, making wSFT a strict generalization of “best-of- n SFT”. Namely, if we weight the best-of- n selection on each $\mathcal{K}_T^{(t)}$ as 1, and all other elements as 0, this loss reduces to:

$$\mathcal{L}_{\text{wSFT}}(\theta) = \mathbb{E}_T \left[-\log G_\theta(y_T \mid \Psi(c_T, x_T, y_{T,0}), \Delta) \right]$$

With this, we train the base iteration t model G_t with $\mathcal{L}_{\text{wSFT}}$ on $\mathcal{S}_{\text{wSFT}}^{(t)}$ to get our iteration t post-wSFT model $G_t^{(\text{wSFT})}$.

4.3.3 IRPO (ITERATIVE REASONING PREFERENCE OPTIMIZATION)

We apply Iterative Reasoning Preference Optimization (IRPO) for proof optimization. IRPO proceeds in rounds: the current model generates multiple candidates per input; those generations are judged to form preference pairs; then a DPO+NLL objective is optimized using the previous iteration’s model as the reference; the newly trained model is used to regenerate pairs at the next round. In the original formulation this judging is typically answer correctness; we instead use the scalar improvement score S_Δ from §3.3 to rank candidates. Additionally, unlike the original IRPO construction, where training is done on both the reasoning trace as well as the final output, we mask the reasoning trace during training to just train on the final output. We ablate this in §5.6 to examine the effects of masking and partially masking the reasoning traces during training.

Now, in order to construct the IRPO pairset, we proceed to post-process $\mathcal{S}_{\text{re}}^{(t)}$ as follows. For a given theorem T , we partition the candidates $\tilde{\mathcal{Y}}_T^{(t)}$ into $\mathcal{K}_T^{(t)} / \sim_s$ and its complement, denoted $\bar{\mathcal{K}}_T^{(t)}$, where $\sim_s$ deduplicates by equal scores $\hat{s}_{T,i} = \hat{s}_{T,j}$. Sort $\mathcal{K}_T^{(t)}$ in descending order by $\hat{s}_{T,i}$.

Then, relative to two hyperparameters $W, L \in \mathbb{N}$, we may define:

$$\begin{aligned} \mathcal{W}_T &= \mathcal{K}_T^{(t)}[:W] \\ \mathcal{L}_T &= \bar{\mathcal{K}}_T^{(t)}[:L] \end{aligned}$$

Where the latter is arbitrarily (deterministically) ordered – say, lexicographically. With this, we form two families of preference pairs (rejected, chosen):

$$\mathcal{P}_T^{\ell \rightarrow w} = \{(b, g) : b \in \mathcal{L}_T, g \in \mathcal{W}_T\}, \quad \mathcal{P}_T^{w \rightarrow w} = \{(g', g) : g, g' \in \mathcal{W}_T, \hat{s}_{T,g} > \hat{s}_{T,g'}\}.$$

And with this, we form our IRPO training dataset as the following set:

$$\mathcal{S}_{\text{IRPO}}^t = \bigcup_{T \in \mathcal{P}_{\text{train}}} (\mathcal{P}_T^{\ell \rightarrow w} \cup \mathcal{P}_T^{w \rightarrow w})$$

Which can be seen elementwise as a collection of tuples $(c_T, x_T, y_{T,0}, y_{T,\ell}, y_{T,w})$ with (rejected, chosen)=(y_ℓ, y_w).

With this dataset, we calculate the IRPO loss on each item T as the (weighted) sum of the DPO loss of the preference pair and the negative log-likelihood (NLL) loss over the winner:

$$\mathcal{L}_{\text{IRPO}}(T) = \mathcal{L}_{\text{DPO}}(y_{T,\ell}, y_{T,w} \mid \Psi(c_T, x_T, y_{T,0}), \Delta) + \alpha \mathcal{L}_{\text{NLL}}(y_{T,\ell}, y_{T,w} \mid \Psi(c_T, x_T, y_{T,0}), \Delta)$$

More explicitly, we have that:

$$\begin{aligned} \mathcal{L}_{\text{IRPO}}(G_\theta; G_{\text{ref}}) &= \mathbb{E}_T \left[-\log \sigma \left(\beta \log \frac{G_\theta(y_{T,w} \mid \Psi(\hat{x}_T), \Delta)}{G_{\text{ref}}(y_{T,w} \mid \Psi(\hat{x}_T), \Delta)} - \beta \log \frac{G_\theta(y_{T,\ell} \mid \Psi(\hat{x}_T), \Delta)}{G_{\text{ref}}(y_{T,\ell} \mid \Psi(\hat{x}_T), \Delta)} \right) \right. \\ &\quad \left. - \frac{\alpha}{|y_{T,\ell}|} \log G_\theta(y_{T,w} \mid \Psi(\hat{x}_T), \Delta) \right] \end{aligned}$$

Where in practice $\hat{x}_T = (c_T, x_T, y_{T,0})$ and $G_{\text{ref}} = G_t^{(\text{wSFT})}$. And after training with respect to this objective, we obtain $G_t^{(\text{IRPO})}$, which we define as G_{t+1} . With this, we may continue to repeat the iteration process.

5 EXPERIMENTS

5.1 SETUP

TODO ACTUALLY FIX UP THIS SSECTION Dataset and split. All results are on the held-out test split $\mathcal{P}_{\text{test}} = \text{MiniCTX v2}$. All training is on the train split $\mathcal{P}_{\text{train}}$ Consisting of the complement of $\mathcal{P}_{\text{test}}$ on each respository. See ?? for dataset details.

Evaluation protocol. Unless noted, we evaluate with pass@16 (best-of-16 sampling; Sec. 3.3) and report the improvement score $S_{\Delta}(c, x; y \| y_0)$ with the tested metric(s) specified for each individual result. We operate with a base model of `DeepSeek-R1-Distill-Qwen-7B`, with **FINISH UP ALL THESE REPRODUCIBILITY DETAILS. INCLUDING GPT-5 VERSION, HYPERPARAMS, READABILITY METRIC HYPERPARAMS ETC.**

Systems compared. *Base model* (iter0), *ImProver² Cold Start* (our expert-iteration without teacher), *GPT-5*, *GPT-5 + Neurosymbolic Augmentation* (our Ψ bundle at inference), and *ImProver² Distill* (iter1 distilled from a teacher, then self-improved).

5.2 MAIN RESULTS ($\mathcal{P}_{\text{TEST}} = \text{MINICTX}$, PASS@16)

Table 1: **Main comparison at pass@16 on $\mathcal{P}_{\text{test}}$ (minictx).** Each cell is the mean improvement vs.

y_0	Model	Length	Modularity	Dependency	Readability
	Base model	0.104	0.291	0.070	0.127
	GPT-5-chat	0.374	0.875	0.069	.
	GPT-5-chat + Neurosymbolic Aug.	0.474	0.953	0.095	.
	GPT-5-mini	0.514	3.68(reward hacking, rerun)	0.225	0.302
	GPT-5-mini + Neurosymbolic Aug.	0.623	3.308(reward hacking, rerun)	0.243	0.308
	ImProver ²	.	.	.	.

Figure placeholder: 2×2 distribution-shift charts for Base, ImProver² Cold Start, and ImProver² Distill across the four metrics (histograms or CDFs).

Figure 2: Distribution shifts on $\mathcal{P}_{\text{test}}$ at pass@16.

Table 2: **Per-iteration improvements.** Each cell is *Cold / Distill NO DISTILL* at pass@16 (improvement vs. y_0).

Iteration	Length	Modularity	Dependency	Readability
Base	0.104	0.425	0.069	0.127
Iter 0.5	0.152	0.389	0.071	0.202
Iter 1	0.276	0.847	0.069	0.343
Iter 1.5	0.100	0.854	0.121	0.349
Iter 2	0.320	0.885	0.137	.
Iter 2.5	.	0.854	0.150	.
Iter 3	.	0.887	0.150	.
Iter 3.5	.	.	0.156	.
Iter 4	.	.	0.193	.

Figure placeholder: 2×2 lines—Cold Start vs. Base vs. GPT-5 across iterations for each metric (best-of-16).

Figure 3: Cold Start iteration curves vs. Base and GPT-5.

5.3 ITERATION STUDY (PASS@16)

5.4 CROSS-BENCHMARK COMPARISON (PASS@16)

5.5 DOWNSTREAM THEOREM PROVING FROM OPTIMIZED CORPORA

5.6 ABLATIONS (PASS@16 UNLESS NOTED)

Neurosymbolic ablations. Evaluate Base with no augmentation, then add one at a time auto-informalization, context slice, and CoS.

Training objective ablation. Compare SFT, wSFT, DPO, IRPO, and their combinations.

Hyperparameters (wSFT + IRPO). Sweep the specific settings you provided.

Frontier ablations. Compare retraining policies that bias toward frontier items with different hardness filters.

Readability study. Visualize the mean variance in scores between identical judges (i.e. how stable is the metric). Optionally have experts judge if A/B is more "readable" (likely not for ICLR)

Our experiments used $k = 3$, $w = (3, 2, 2, 2, 4, 1, 6)$.

TODO

Figure placeholder: layered line plot of BoN performance on ImProver benchmark

Figure 4: Direct comparison of ImProver vs ImProver² (ImProver benchmark, on length metric)

Table 3: **Training a prover by SFT on different corpora.** Evaluate pass@16 and pass@64 on a held-out proving benchmark.

Training data	pass@16	pass@64
SFT on train set (original)	.	.
SFT on <i>readable</i> train set	.	.
SFT on <i>modular</i> train set	.	.

5.7 QUALITATIVE

6 CONCLUSION

AUTHOR CONTRIBUTIONS

If you’d like to, you may include a section for author contributions as is done in many journals. This is optional and at the discretion of the authors.

ACKNOWLEDGMENTS

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

REFERENCES

A APPENDIX

You may include other additional sections here.

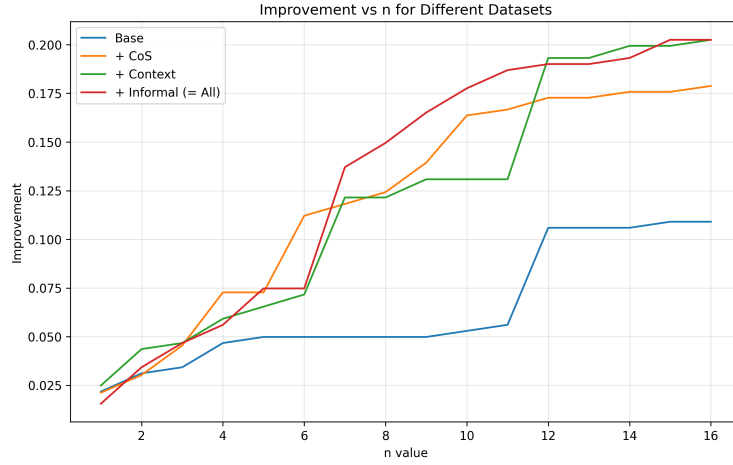


Figure 5: Neurosymbolic ablations on length metric.

Table 4: Training algorithms on Base and Iter 1.

Method	Iter 1	Iter 2
SFT	0.103	0.187
SFT + DPO	0.215	0.215
wSFT	0.152	.
wSFT + DPO	0.267	0.276
wSFT + IRPO	0.276	0.320

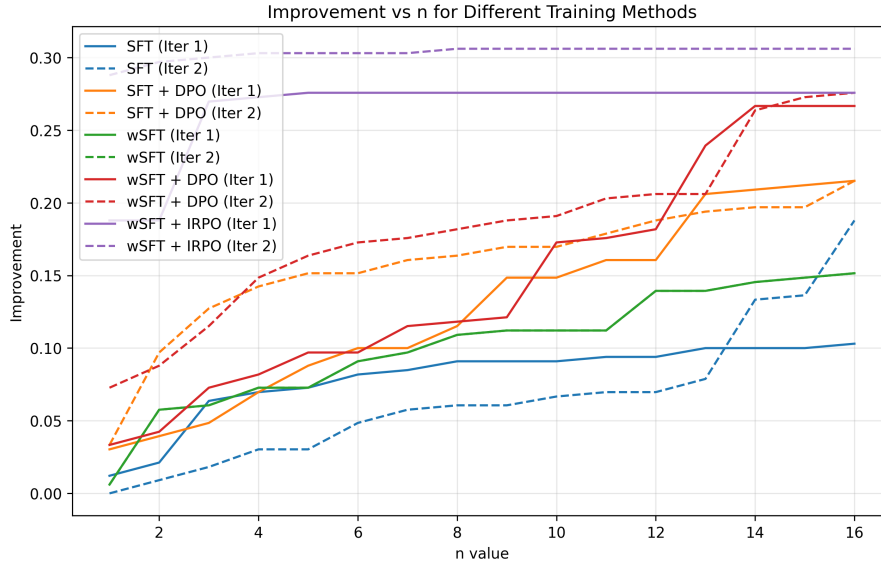


Figure 6: Direct comparison of SFT,wSFT,DPO,IRPO performance on iteration 1,2.

Table 5: wSFT+IRPO hyperparameter sweeps. Each row is a separate run.

Var. cap (%)	α	Iter 1
100	1.0	.
80	1.0	0.276
50	1.0	.
80	0.6	0.236
80	1.0	0.276
80	2.0	0.154

Table 6: **IRPO** (W, L) **hyperparameter sweeps**. Each row is a separate run.

Policy (W, L)	Iter 1
(1, 1)	0.185
(3, 2)	0.136
(16, 1)	0.176
(1, 16)	0.242
(16, 16)	0.276

Table 7: **Replay strategies across iterations (wSFT + DPO)**. Each cell shows the metric improvement; columns group by iteration.

Policy	Base	iter1	iter2-wSFT	iter2
No replay	0.104	0.266	0.084	0.091
20% mark	0.104	0.266	0.085	0.182
20% join	0.104	0.266	0.121	0.263
20% replace	0.104	0.266	0.1	0.275
40% mark	0.104	0.266	0.087	0.182
40% join	0.104	0.266	0.072	0.182
40% replace	0.104	0.266	0.090	0.275

Table 8: **Frontier selection**. Base/Iter 1/Iter 2 under three frontier policies.

Policy	iter1	iter2-wSFT	iter2
Base retrain	0.276	0.127	0.273
Frontier (IR < 0.5)	0.276	0.100	0.320

Figure placeholder: side by side examples of unoptimized/optimized theorems

Figure 7: Qualitative example